Implementing Stack Data Structure

In this module, we'll cover how to implement a stack data structure using different programming languages and data structures. This is where you'll see how powerful, versatile, and useful stacks can be!

Stack Implementation Using an Array

Implementing a stack using an array is one of the most straightforward ways. Let's see how you can do this in different programming languages:

Java

```java
import java.util.EmptyStackException;

public class Stack<T> {
    private Object[] stack; // An array to store the elements of the stack
    private int top; // An integer to keep track of the top element's index

    // Constructor to initialize the stack with a specified size
    public Stack(int size) {
        stack = new Object[size]; // Create a new array of objects with the given size
        top = -1; // Initialize the top index to -1, indicating an empty stack
    }

    // Method to push an item onto the stack
    public void push(T item) {
        if (top == stack.length - 1) {
            throw new IndexOutOfBoundsException("Stack is full"); // Check if the stack
        }
        stack[++top] = item; // Increment the top index and add the item to the stack
    }

    // Method to pop and remove the top item from the stack
```

The primary advantage of using an array for implementing a stack is that it's simple and requires no additional setup. However, the size of the array can limit the stack size, leading to a stack overflow.

Stack Implementation Using Linked List

An alternative way of implementing a stack is by using a linked list. This method can overcome the size limitation issue present in the array implementation. The code will look something like this:

Java

```java
import java.util.EmptyStackException;

public class Stack<T> {
    // Private inner class representing a Node in the stack
    private static class Node<T> {
        private T data;         // Data stored in the node
        private Node<T> next;   // Reference to the next node in the stack

        // Constructor to create a new node with the given data
        private Node(T data) {
            this.data = data;
        }
    }

    private Node<T> top;  // Reference to the top node of the stack

    // Pop operation: Removes and returns the top element of the stack
    public T pop() {
        // Check if the stack is empty
        if (top == null) {
            throw new EmptyStackException(); // Throw an exception if it's empty
```

The key difference here is that we're using a Node class to create nodes and linking them together to form a stack. This method allows our stack to be dynamically sized, avoiding the overflow issue we saw with arrays.

Here is how different programming languages implement the stack class:

Language	API
Java	java.util.Stack
Python	Implemented through List
C++	std::stack
JavaScript	Implemented through Array